

CSG3101 Applied Project

Technical Report For:

Intelligent Spatial Management System

Team Members:

Sanuka Bandara Wasala(Team Leader):10665590

Minaga Ransilu Piyadigama:10659027

Chandupa Poorna Lokuliyana:10664773

Mohammed Shiham Shakoor:10647877

Mahipala Mudalige Tharushi Umesha:10676110

Project Supervisor: Inoshi Jayaweera
Lecturer: Ann Appuhamy

Contents

Table of Figures	3
Introduction.....	4
Executive summary.....	4
Scope.....	5
In Scope	5
Out of Scope	6
System Design Overview.....	6
Architectural Design	6
Technology Stack.....	7
Data Flow Diagram (DFD)	7
Level 1: High-Level Data Flow	7
Level 2: Detailed Breakdown	7
APIs Used in the System.....	7
Software Requirement Specification (SRS).....	9
Assumptions.....	9
Requirements	9
Functional Requirements	9
Non-Functional Requirements	10
Use case Diagram	12
Login.....	13
Action Flow	13
Alternative Action Flow.....	13
User Registration	14
Action Flow	14
Alternative Action Flow.....	14
Room Booking.....	15
Action Flow	15
Alternative Action Flow.....	16
Cancel Booking.....	16
.....	16
Action Flow	17
Alternative Action Flow.....	17

Penalty Enforcement.....	17
Action Flow	18
Alternative Action Flow.....	18
Generate Analytics Report	18
Action Flow	19
Alternative Action Flow.....	19
Room Availability Check.....	19
Action Flow	19
Alternative Action Flow.....	20
Check-In.....	20
Action Flow	20
Alternative Action Flow.....	21
Feedback Submission.....	21
Action Flow	22
Alternative Action Flow.....	22
Penalty Management.....	22
Action Flow	23
Alternative Action Flow.....	23
Admin - Manage Rooms.....	23
Action Flow	24
Alternative Action Flow.....	24
Admin-Add a Room.....	24
Action Flow	25
Alternative Action Flow.....	25
Admin - Generate Reports	26
Action Flow	26
Alternative Action Flow.....	26
Chatbot Interaction.....	27
Action Flow	27
Alternative Action Flow.....	27
Admin-Deactivate User.....	28
Action Flow	28
Alternative Action Flow.....	29

Admin-Create Notification	29
Action Flow	30
Alternative Action Flow.....	30
Handling No-show or Late Cancellation Penalty.....	31
Action Flow	31
Alternative Action Flow.....	32
Overall - Class Diagram.....	32
API Specifications	33
Users App.....	33
Rooms App.....	33
Bookings App.....	33
Chatbot App	33
Feedback App.....	33
Notifications App	33
Penalties App	34
Analytics App.....	34
Conclusion	34
Technical Achievements	34
Challenges Addressed	35
Key Findings.....	35
Future Technical Enhancements	35
References.....	36

Table of Figures

Figure 1	12
Figure 2	13
Figure 3	14
Figure 4.....	14
Figure 5	15
Figure 6	16
Figure 7	17
Figure 8.....	18
Figure 9	19

Figure 10 20

Figure 11 21

Figure 12 22

Figure 13 23

Figure 14 24

Figure 15 26

Figure 16 27

Figure 17 28

Figure 18 29

Figure 19 31

Figure 20 32

Introduction

The **Room Booking System (RBS)** is a modern, web-based application designed to address the growing challenges of resource management at **ECU Sri Lanka (ECUSL)**. As the campus population expands and demand for shared facilities like study rooms and lecture halls increases, traditional manual booking processes have proven to be inefficient and prone to errors. This system offers a centralized, scalable, and user-friendly platform to streamline the room reservation process for students, lecturers, and administrators.

The RBS provides a comprehensive solution with features such as **real-time room availability tracking**, **role-based access control**, and **penalty enforcement mechanisms** to encourage responsible resource usage. It also includes **notification systems** to keep users updated about their bookings and **analytics dashboards** to provide administrators with actionable insights into room utilization trends.

Developed using state-of-the-art technologies, the RBS ensures robustness, scalability, and adaptability for future enhancements such as multi-campus support and mobile app integration. By automating complex administrative tasks and improving user experience, the Room Booking System is a significant step towards efficient campus resource management, fostering a better academic and operational environment at ECUSL.

Executive summary

The web-based Room Booking System (RBS) optimizes administrative processes which manage shared campus facilities utilized at ECU Sri Lanka (ECUSL). Studies have shown that traditional manual booking methods now face performance challenges when managing study rooms and lecture halls thus creating issues with resource scheduling and administrative costs. The RBS offers students lecturers and administrators a centralized platform alongside user-friendly features and infinite scalability which optimizes facility booking procedures. The platform features real-time space scheduling visibility alongside access permissions management and booking punishment infrastructure alongside automatic notification services for user updates about reservations. Platform dashboards present administrative analytics for helping administrators reach decisions through data examination while maximizing resource efficiency. The implementation of a chatbot system enhances user satisfaction through automatic assessments for room availability and booking management queries. The technological framework that underpins the RBS incorporates new application development standards and implements an MVC (Model-View-Controller) structure for client-server interactions with HTML, CSS, JavaScript as

frontend elements and Python Django and MySQL used for backend operations and storage respectively. Communication between frontend and backend components runs efficiently through the RESTful APIs and security is maintained by implementing JWT-based authentication and HTTPS protocols.

Scope

The **Room Booking System (RBS)** is designed to simplify and optimize the process of reserving campus facilities at **ECU Sri Lanka (ECUSL)**. It serves as a centralized platform for students, lecturers, and administrators to manage room bookings efficiently while ensuring fair resource allocation and transparency. By automating complex manual processes and integrating advanced features, the system provides a seamless and structured approach to campus room management.

In Scope

1. **User Registration and Authentication:**
 - Secure registration for new users using email addresses and password-based authentication.
 - Role-based access control for students, lecturers, and administrators, ensuring appropriate feature access.
2. **Room Reservation and Management:**
 - Real-time tracking of room availability, allowing users to view and reserve rooms based on specific requirements such as location, capacity, and time slots.
 - Dynamic room approval processes based on administrator-defined policies.
 - Administrators can create, edit, and manage room details, including capacity, availability, and booking restrictions.
3. **Penalty Management:**
 - Automated tracking and enforcement of penalties for no-shows, late cancellations, and rule violations to ensure accountability.
 - Integration of a penalty resolution workflow to manage user disputes.
4. **Notifications:**
 - Automated email and in-app notifications for booking confirmations, cancellations, and penalties.
 - Notifications for administrators to review pending room approvals and monitor violations.
5. **Analytics and Reporting:**
 - Advanced analytics dashboards for administrators to monitor room utilization trends, identify peak usage hours, and detect underutilized resources.
 - Exportable reports in formats such as CSV or PDF to support administrative decision-making.
6. **Feedback Collection:**
 - Enable users to submit feedback regarding room quality, availability, or booking experience.
 - Integration of sentiment analysis to identify areas for improvement.
7. **Chatbot Integration:**
 - AI-powered chatbot to assist users with room availability queries, booking workflows, and system navigation.
8. **Testing and Quality Assurance:**

- Comprehensive testing, including functional, integration, and performance testing, to ensure system reliability and robustness.
 - User acceptance testing (UAT) to gather feedback and ensure alignment with stakeholder needs.
9. **Documentation:**
- Preparation of detailed project documentation, including requirements specification, system architecture, user guides, and test reports.

Out of Scope

1. **Integration with External Systems:**
 - The system does not rely on external room scheduling tools or third-party datasets.
2. **Mobile Application Development:**
 - A mobile application version is not included in the current scope but may be considered for future enhancements.
3. **Third-Party Feedback Analysis:**
 - Feedback collection is limited to user inputs within the system, with no integration of third-party sentiment analysis tools.
4. **Complex Financial Transactions:**
 - The system does not handle financial transactions for room bookings or penalty payments, focusing instead on administrative enforcement.

System Design Overview

Architectural Design

The Room Booking System (RBS) follows a **client-server architecture** with an **MVC (Model-View-Controller) design pattern** implemented using Django. The high-level architecture includes:

1. **Frontend (Client-Side)**
 - Users interact with the system via a web interface built using HTML, CSS, and JavaScript.
 - Handles user inputs, validation, and requests to the backend via RESTful APIs.
2. **Backend (Server-Side)**
 - Developed using Python Django framework.
 - Manages business logic, database interactions, and API services.
 - Ensures data processing, authorization, and validation.
3. **Database Layer**
 - MySQL Workbench is used to store user data, room details, bookings, and penalties.
 - Structured schema with tables like users, rooms, bookings, and penalties.
4. **External Tools**

- **Project Management:** ClickUp for task tracking and Microsoft Teams for collaboration.
- **Testing:** Postman for API testing.
- **Deployment:** Hosted on campus servers with potential for cloud migration.

Technology Stack

Component	Technology Used
Frontend	HTML, CSS, JavaScript
Backend	Python Django (Django REST Framework)
Database	MySQL Workbench
API Testing	Postman
Project Management	ClickUp, Microsoft Teams
Authentication	JWT (JSON Web Token)

Table 1

Data Flow Diagram (DFD)

Level 1: High-Level Data Flow

1. User submits booking requests via the frontend.
2. The frontend sends a request to the backend API.
3. The backend processes the request, verifies data, and interacts with the database.
4. The response is sent back to the frontend for user feedback.

Level 2: Detailed Breakdown

1. **User Registration/Login**
 - User enters credentials → API validates → Response (success/fail)
2. **Room Booking Process**
 - User selects room → API checks availability → Booking confirmation
3. **Penalty Enforcement**
 - System monitors no-shows → Applies penalties → Notifies user
4. **Feedback Submission**
 - User submits feedback → API processes data → Stores in the database

APIs Used in the System

The Room Booking System depends on APIs (Application Programming Interfaces) to serve as intermediaries that enable frontend applications to talk with backend components. The API endpoints enable diverse operations from user login management to booking appointments through notification procedures that stabilize user system interaction.

1. User Authentication API

During login the User Authentication API performs the task of authenticating user credentials. When users register they can access the system through entering their valid email followed by their password. Following successful user verification the application generates security tokens which function as JWTs to prove authorized access requests.

2. User Profile API

Available data about the current user profile exists through operations accessible via the User Profile API. The User Profile API allows access to individual user information through role determination together with profile images and personal contact details for experience administration and proper role-based access safeguards in the system.

3. Room Details API

The Room Details Application Programming Interface serves up system data on available rooms. The API enables users to get detailed room characteristics including space availability together with location information and current booking status. Such API demonstrates its importance by displaying up-to-the-minute space availability and helping users with their room selection based on their personal preferences.

4. Booking API

Through the Booking API users create new room reservations. When booking a reservation through the system users must provide necessary details including the selected room and booking date together with start and end timing and reason for their reservation. The application programming interface (API) verifies booking requests with proper validation and produces system records of bookings with automated success notification to users.

5. Booking Cancellation API

Through the Booking Cancellation API platform users possess the ability to end reservations they created beforehand provided the cancellation schedule permits it. This API verifies booking data before processing cancellation operations and applies system updates to room availability status. The API enhances room availability by creating availability in unused bookings for new users.

6. Notification API

The Notification API delivers important messages to users regarding their bookings together with penalty combinations and system alert messaging functions. Through this API the system delivers both email alerts and in-app notifications to users concerning their booking confirmations and subsequent cancellations as well as penalty implementations. The system maintains a process which continuously keeps users updated about all their system activities.

7. Error Handling in APIs

The API responses implement detailed error detection systems. The system provides complete API error management for failed requests through the use of suitable response messages which handle account authentication token expiration and unauthorized access attempts and incorrect API requests. The frontend performs targeted actions that range from user redirection to error message display following API deployment failures.

Conclusion

Through these APIs the Room Booking System establishes a central communication platform which maintains secure user-system interactions. The API system enables proper reservation management along with notification execution and user profile maintenance to deliver an unbroken user journey.

Software Requirement Specification (SRS)

The **Software Requirement Specification (SRS)** details the system's functional and non-functional requirements to ensure the **Room Booking System (RBS)** meets the needs of its stakeholders. It outlines the assumptions, functional requirements, and technical constraints considered during development.

Assumptions

1. **User Access:**
 - Users will access the system through a web browser.
 - Internet connectivity is required to access the platform.
2. **User Roles:**
 - The system supports three primary roles: **Student**, **Lecturer**, and **Administrator**.
 - Role-based access control ensures that users can only access features appropriate to their role.
3. **Room Availability:**
 - Room details (capacity, location, and availability) are pre-configured by administrators.
4. **Notifications:**
 - Email services are functional, and users have valid email addresses to receive notifications.
5. **Penalties:**
 - Penalty rules for no-shows or late cancellations are predefined and enforced automatically.
6. **Data Backup:**
 - The system performs regular automated backups of critical data.

Requirements

Functional Requirements

FR-1: Registered users should be able to log in to the system, and the system should validate the credentials.

FR-2: Unregistered users should be able to sign up for the system using their email and password.

FR-3: Students and lecturers should be able to view available rooms in real time.

FR-4: Users should be able to book rooms by selecting the room, purpose, and preferred time slot.

FR-5: Users should be able to cancel their bookings within allowed timeframes.

FR-6: Administrators should be able to create, edit, and delete room details, including capacity, location, and availability.

FR-7: Administrators should have the ability to approve or reject room booking requests.

FR-8: Penalty rules for no-shows or late cancellations should be automatically applied by the system.

FR-9: The system should send email and in-app notifications for booking confirmations, cancellations, approvals, and penalties.

FR-10: Administrators should be able to view analytics and generate reports on room utilization trends.

FR-11: The system should provide role-based access control to restrict features based on user roles (Student, Lecturer, Administrator).

FR-12: Users should be able to submit feedback on rooms or booking experiences.

FR-13: Sentiment analysis should be applied to feedback to provide actionable insights to administrators.

FR-14: The system should allow administrators to resolve disputes related to penalties.

FR-15: The system should support a chatbot to assist users with queries related to room availability, bookings, and navigation.

Non-Functional Requirements

Performance:

- The system should respond to room availability queries within **500ms** under normal load conditions.
- Support concurrent usage by at least **500 active users** without performance degradation.

Security:

- Ensure secure user authentication using encrypted password storage.
- Implement **HTTPS** for secure communication between the client and server.

Usability:

- Provide a responsive design compatible with mobile and desktop browsers.

- Ensure intuitive navigation for users with clear prompts and error messages.

Scalability:

- Support future expansion to include multi-campus operations.
- Enable seamless integration with potential mobile applications.

Maintainability:

- Modular architecture to facilitate updates and new feature integrations.
- Maintain comprehensive logging for debugging and monitoring.

Reliability:

- Ensure **99.9% uptime** during operational hours.
- Include automated data backup functionality with daily schedules.

Use case Diagram

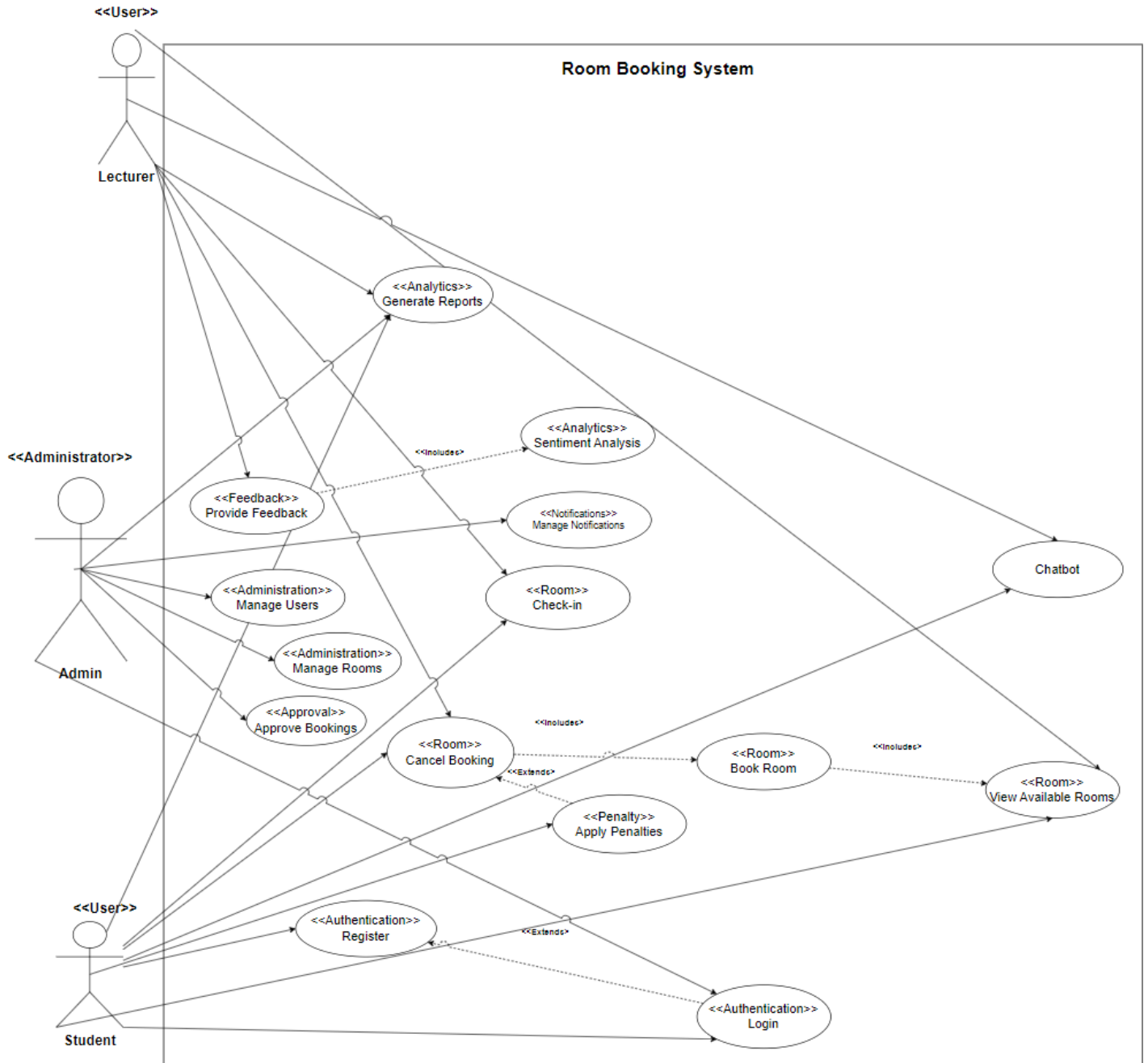


Figure 1

Login

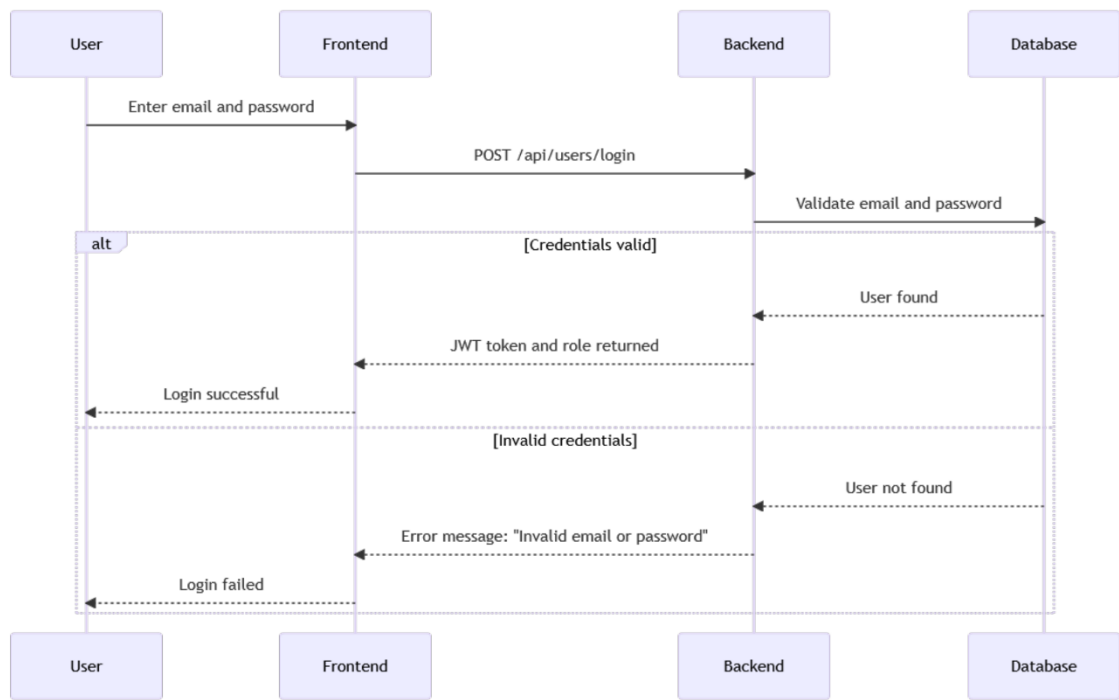


Figure 2

Use Case ID	1
Use Case Name	Login
Description	Registered users can access the system by entering their respective credentials.
Primary Actor	Lecturer, Student
Secondary Actor	Admin
Requirements	The user should be registered in the system.
Post Action	The user will be granted access to their respective dashboard if their username and password are correct.

Table 2

Action Flow

Step	Action
1	The user is prompted to enter their email and password.
2	The user enters the correct email and password.
3	The system verifies the entered credentials.
4	If the credentials are correct, the user is granted access and redirected to their respective dashboard.

Table 3

Alternative Action Flow

Step	Action
------	--------

1	If the entered credentials are incorrect, the user is prompted to re-enter the required credentials.
2	If the user forgets the password, they are requested to re-register or reset their credentials.

Table 4

User Registration

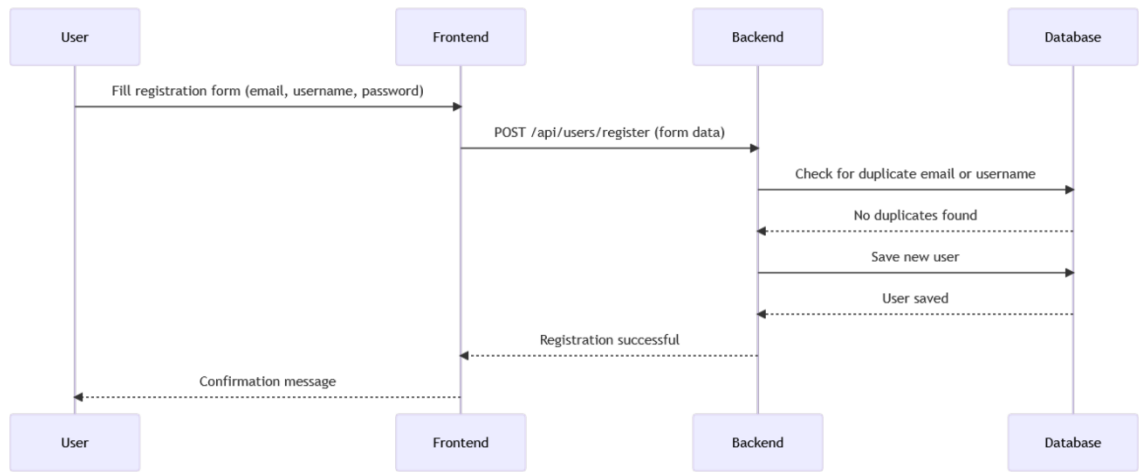


Figure 4

Use Case ID	2
Use Case Name	User Registration
Description	New users can register in the system by providing their email, password, and role (Student, Lecturer).
Primary Actor	Student, Lecturer
Secondary Actor	None
Requirements	The user should provide a unique email address and valid details.
Post Action	The system registers the user and grants access after authentication.

Table 5

Action Flow

Step	Action
1	The user accesses the registration form.
2	The user provides their email, password, and selects their role.
3	The system validates the entered details.
4	The system sends a confirmation message upon successful registration.

Table 6

Alternative Action Flow

Step	Action
1	If the email is already registered, the user is notified and asked to use a different email.

2	If any required fields are missing, the user is prompted to complete the form.
---	--

Table 7

Room Booking

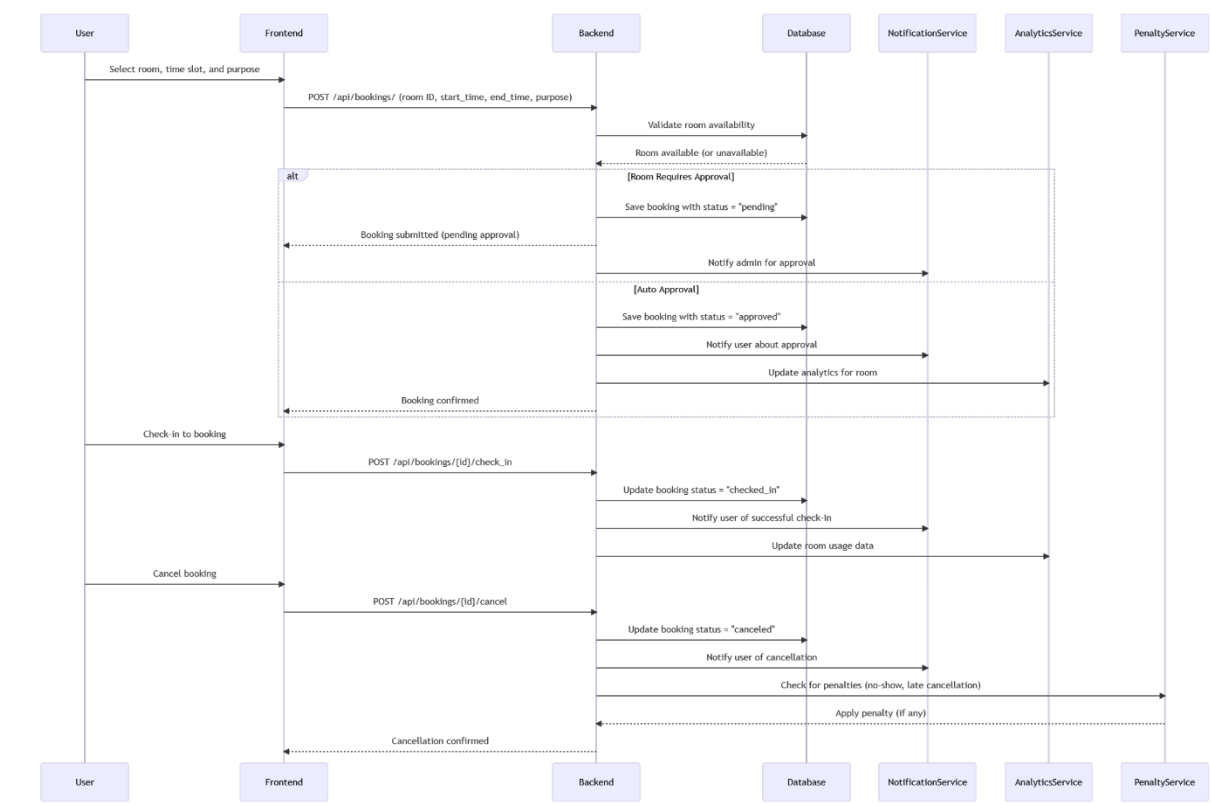


Figure 5

Use Case ID	3
Use Case Name	Room Booking
Description	Users can book available rooms by providing the required details such as time slots and purpose.
Primary Actor	Student, Lecturer
Secondary Actor	None
Requirements	The room should be available for the selected time slot.
Post Action	The system reserves the room and notifies the user about the booking confirmation.

Table 8

Action Flow

Step	Action
1	The user browses available rooms and selects a preferred room.

2	The user provides the booking details, including time slot and purpose.
3	The system validates room availability.
4	The booking is created, and the user is notified of the confirmation.

Table 9

Alternative Action Flow

Step	Action
1	If the room is unavailable, the user is notified to select a different room or time slot.
2	If the user provides incomplete details, they are prompted to complete the form.

Table 10

Cancel Booking

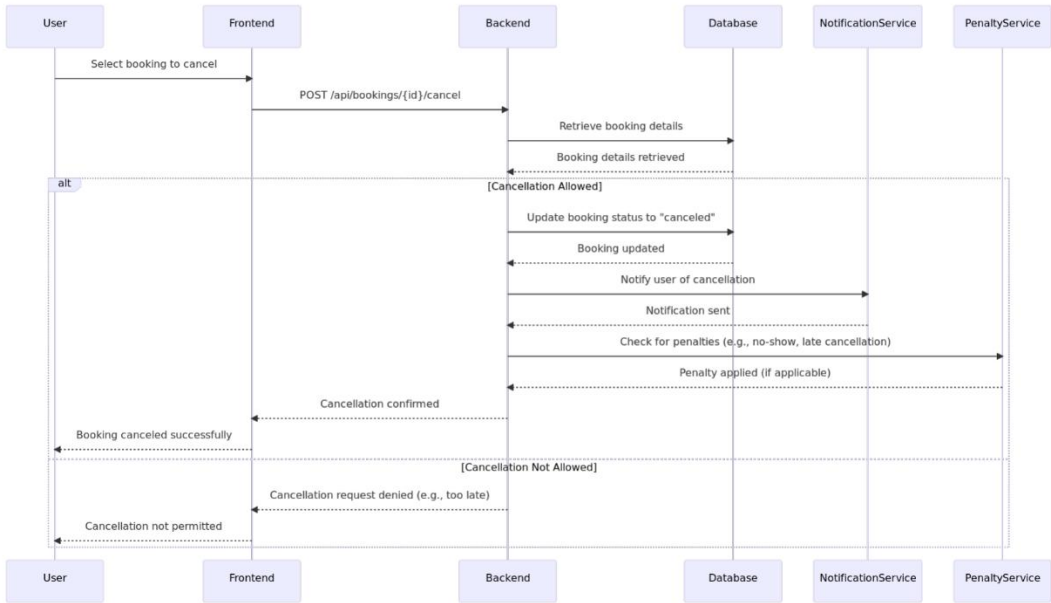


Figure 6

Use Case ID	4
Use Case Name	Cancel Booking
Description	Users can cancel an existing booking within the allowed timeframe.
Primary Actor	Student, Lecturer
Secondary Actor	None
Requirements	The booking should exist, and cancellation should be within the allowed time.
Post Action	The booking is canceled, and the room is released for other users.

Table 11

Action Flow

Step	Action
1	The user accesses their booking history.
2	The user selects a booking to cancel.
3	The system validates the request and cancels the booking.
4	The user receives a cancellation confirmation.

Table 12

Alternative Action Flow

Step	Action
1	If the cancellation timeframe has passed, the user is notified, and no action is taken.

Table 13

Penalty Enforcement

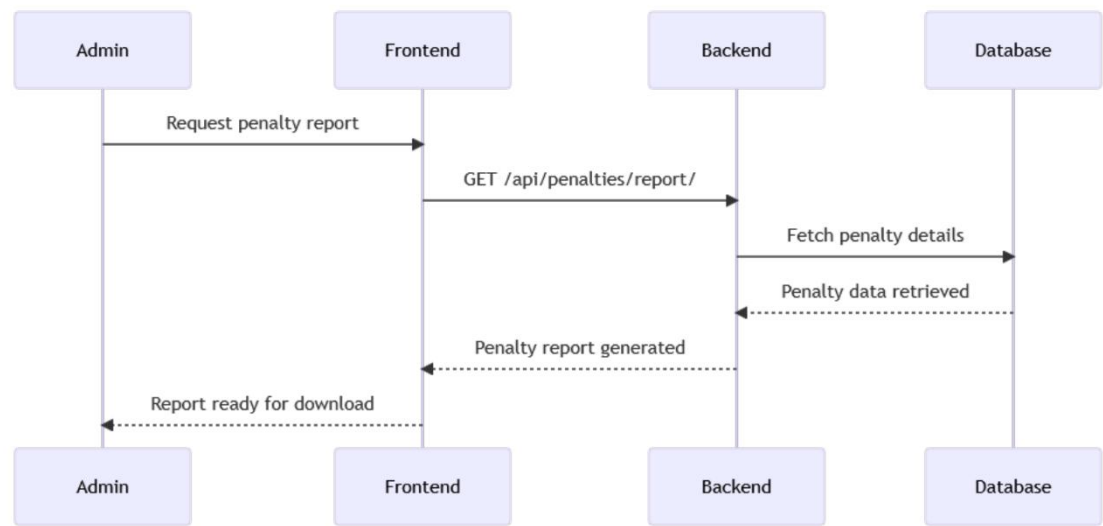


Figure 7

Use Case ID	5
Use Case Name	Penalty Enforcement
Description	The system enforces penalties for no-shows or late cancellations.
Primary Actor	System
Secondary Actor	Admin
Requirements	The penalty rules must be pre-configured in the system.

Post Action	The user is notified of the penalty.
-------------	--------------------------------------

Table 14

Action Flow

Step	Action
1	The system identifies a no-show or late cancellation.
2	A penalty is calculated based on the rules.
3	The penalty is applied to the user’s account.
4	The user is notified about the penalty.

Table 15

Alternative Action Flow

Step	Action
1	If the user disputes the penalty, an admin resolves the case.

Table 16

Generate Analytics Report

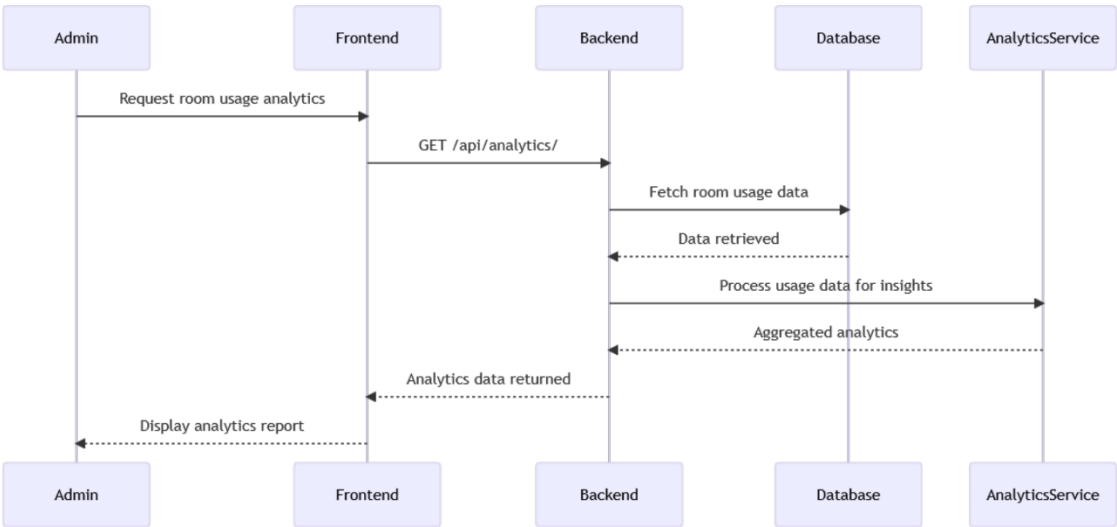


Figure 8

Use Case ID	6
Use Case Name	Generate Analytics Report
Description	Administrators can generate reports on room utilization and booking trends.
Primary Actor	Admin
Secondary Actor	None
Requirements	The system must have sufficient historical booking data.
Post Action	The report is generated and made available for download.

Table 17

Action Flow

Step	Action
1	The admin selects the analytics report feature.
2	The system retrieves and processes room usage data.
3	A report is generated and made available for download.

Table 18

Alternative Action Flow

Step	Action
1	If no data is available, the admin is notified, and no report is generated.

Table 19

Room Availability Check

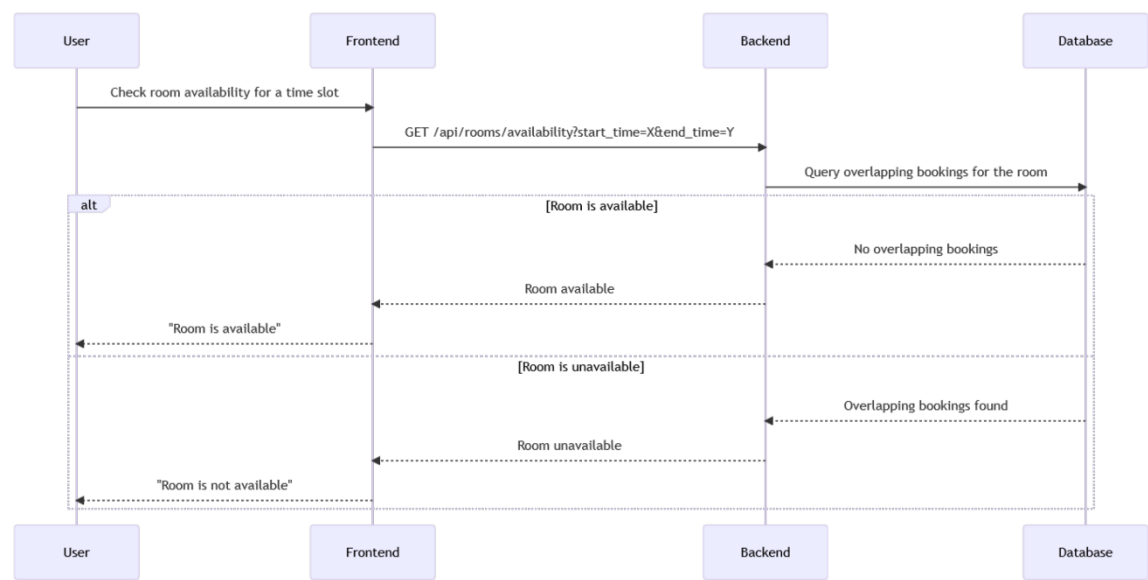


Figure 9

Use Case ID	7
Use Case Name	Room Availability Check
Description	Users can check the availability of rooms for a specific time slot.
Primary Actor	Student, Lecturer
Secondary Actor	None
Requirements	The room data must be up-to-date in the system.
Post Action	The system displays the availability status of the selected room.

Table 20

Action Flow

Step	Action
1	The user inputs the desired time slot and room preference.

2	The system retrieves the room's booking details for the specified time.
3	The system determines whether the room is available.
4	The availability status is displayed to the user.

Table 21

Alternative Action Flow

Step	Action
1	If the room is unavailable, the system suggests alternative time slots or rooms.
2	If the user does not select a valid time slot, they are prompted to correct the input.

Table 22

Check-In

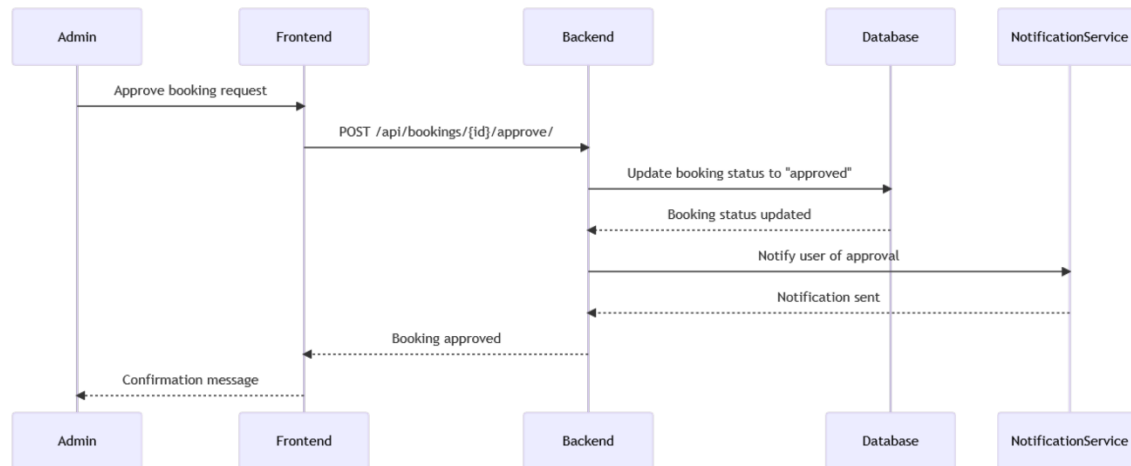


Figure 10

Use Case ID	8
Use Case Name	Check-In
Description	Users can mark their attendance by checking into their reserved room.
Primary Actor	Student, Lecturer
Secondary Actor	None
Requirements	The booking must be approved, and the check-in must occur within the scheduled time.
Post Action	The system updates the booking status to "checked_in."

Table 23

Action Flow

Step	Action
------	--------

1	The user selects the booking they want to check into.
2	The system validates the booking status and timing.
3	The system updates the booking status to “checked_in.”
4	The user receives confirmation of a successful check-in.

Table 24

Alternative Action Flow

Step	Action
1	If the user attempts to check in outside the scheduled time, the request is denied.
2	If the booking status is not “approved,” the check-in is not permitted.

Table 25

Feedback Submission

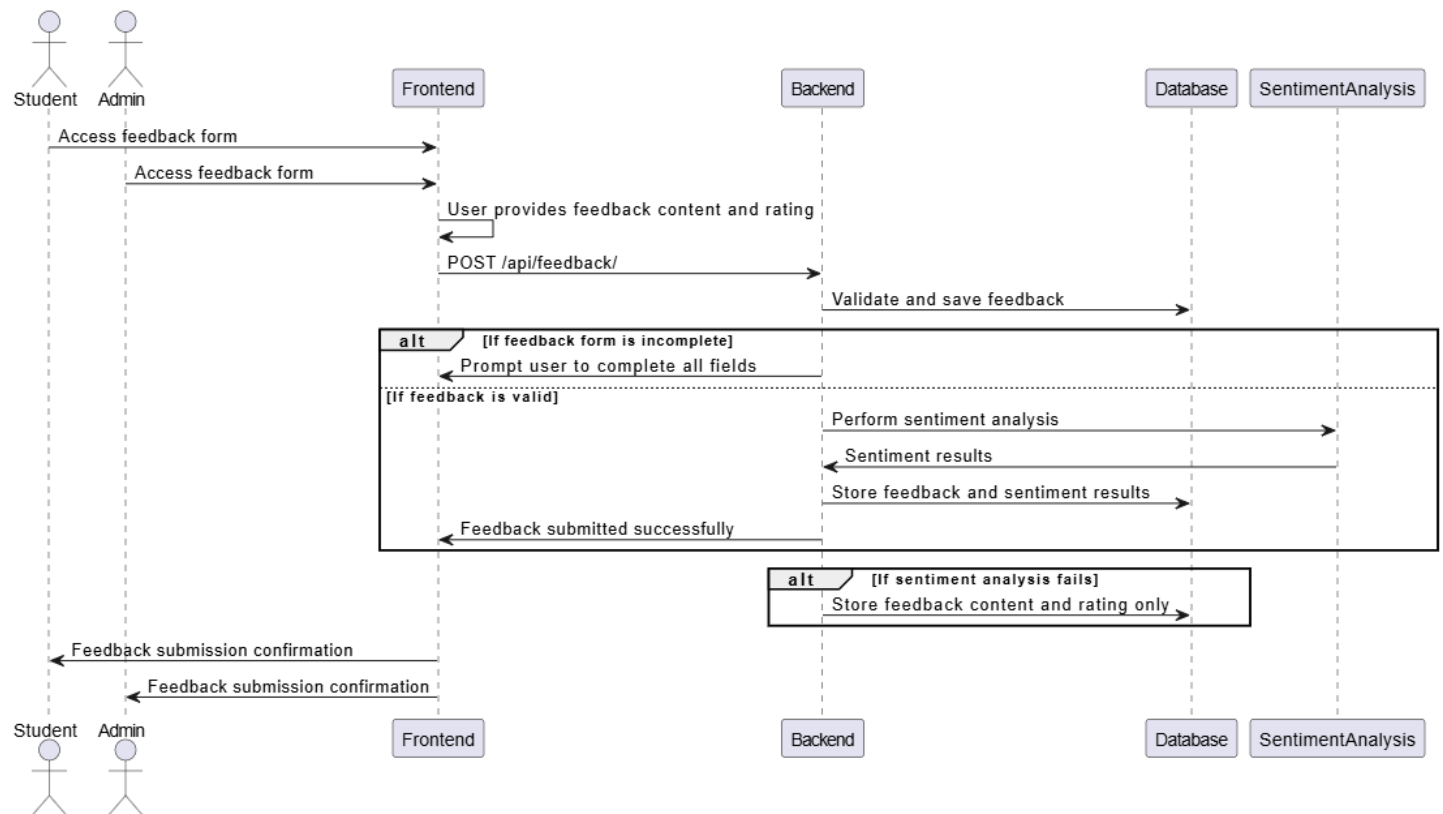


Figure 11

Use Case ID	9
Use Case Name	Feedback Submission
Description	Users can submit feedback on room quality or booking experiences.
Primary Actor	Student, Lecturer
Secondary Actor	Admin

Requirements	The user must have an active booking or experience with the room to submit feedback.
Post Action	The system saves the feedback and performs sentiment analysis.

Table 26

Action Flow

Step	Action
1	The user accesses the feedback form.
2	The user provides feedback content and a rating.
3	The system validates the feedback and saves it in the database.
4	Sentiment analysis is performed on the feedback content.
5	The feedback and sentiment results are stored for admin review.

Table 27

Alternative Action Flow

Step	Action
1	If the feedback form is incomplete, the user is prompted to complete all fields.
2	If sentiment analysis fails, the system saves only the feedback content and rating.

Table 28

Penalty Management

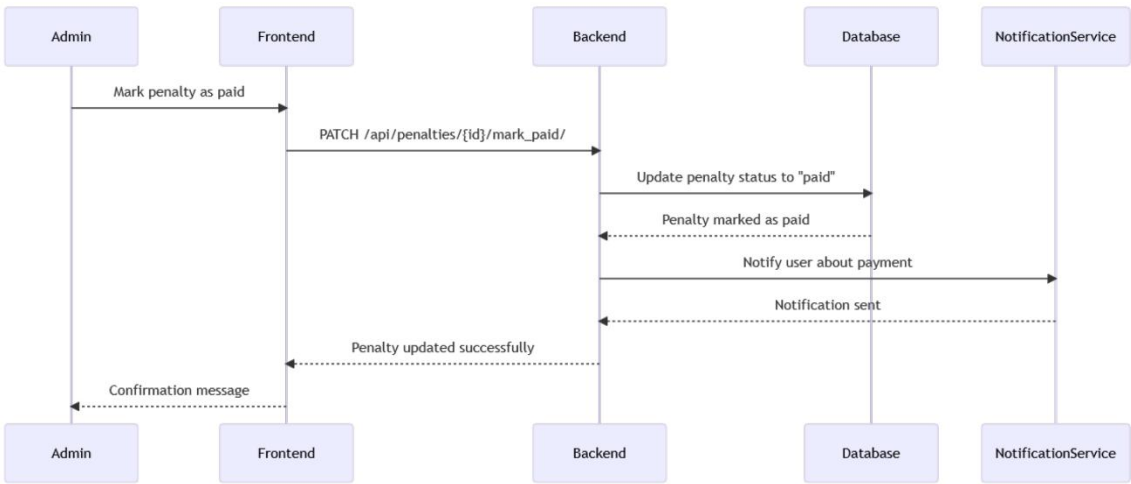


Figure 12

Use Case ID	10
Use Case Name	Penalty Management
Description	The system enforces penalties for no-shows or late cancellations.
Primary Actor	System
Secondary Actor	Admin
Requirements	Penalty rules must be pre-configured in the system.
Post Action	The penalty is applied to the user account, and a notification is sent.

Table 29

Action Flow

Step	Action
1	The system detects a no-show or late cancellation.
2	Penalty rules are applied to calculate the penalty amount.
3	The penalty is recorded in the user’s account.
4	The user receives a notification about the penalty.

Table 30

Alternative Action Flow

Step	Action
1	If the penalty calculation fails, the admin is alerted to resolve the issue manually.

Table 31

Admin - Manage Rooms

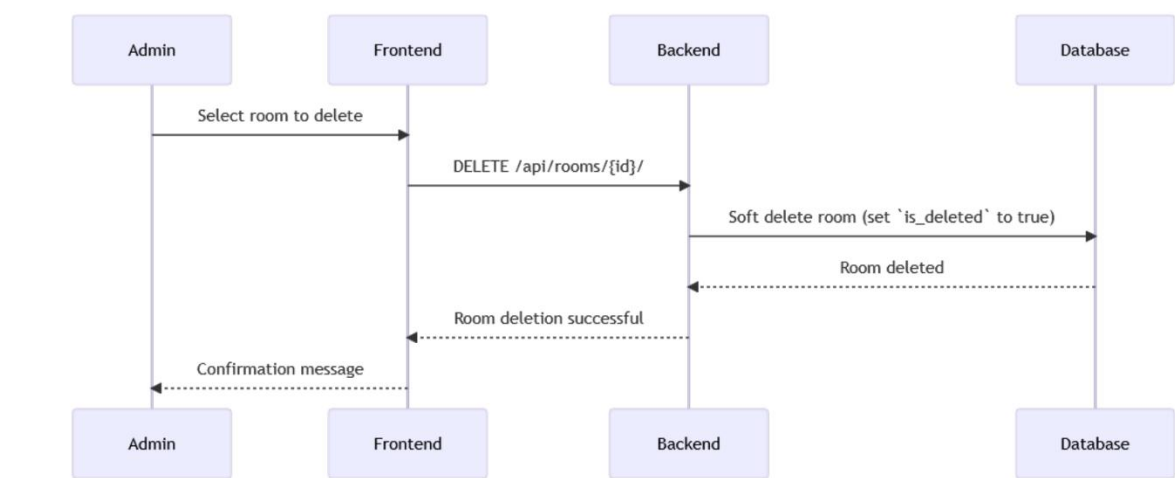


Figure 13

Use Case ID	11
Use Case Name	Manage Rooms
Description	Administrators can create, edit, or delete room records.

Primary Actor	Admin
Secondary Actor	None
Requirements	Admin privileges are required to access this feature.
Post Action	The system updates the room details based on the admin’s action.

Table 32

Action Flow

Step	Action
1	The admin selects the room management module.
2	The admin performs one of the following actions:

Table 33

- Create a new room record.
- Edit details of an existing room.
- Soft delete a room by marking it as unavailable. || 3 | The system validates and updates the room data. || 4 | A confirmation message is displayed. |

Alternative Action Flow

Step	Action
1	If the admin inputs invalid data, the system prompts for corrections.

Table 34

Admin-Add a Room

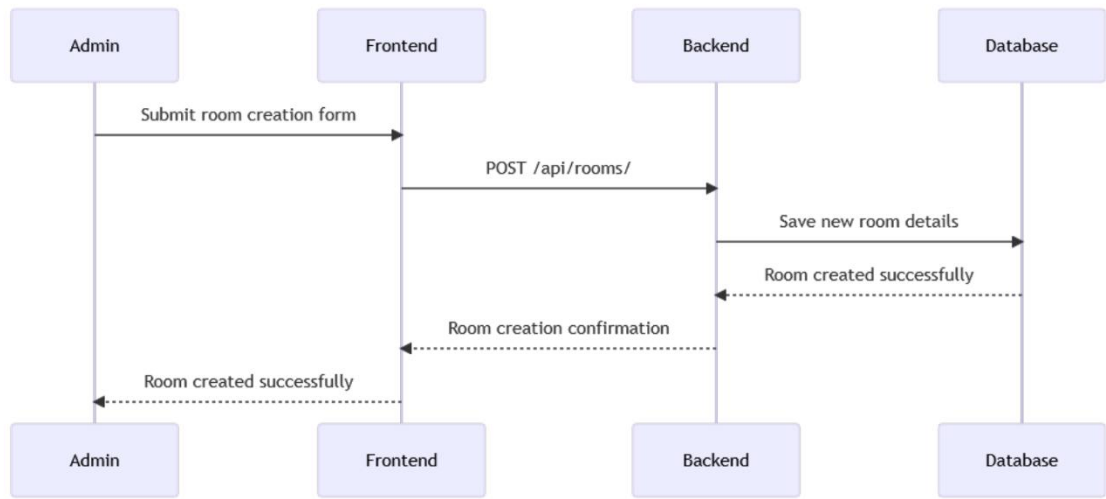


Figure 14

Use Case ID	12
Use Case Name	Admin Add a Room
Description	Admin can add new study rooms to the system.
Primary Actor	Admin
Secondary Actor	None
Requirements	The admin must provide valid room details (name, capacity, availability, etc.).
Post Action	The new room is added to the system and made available for booking.

Table 35

Action Flow

Step	Action
1	The admin submits the room creation form.
2	The frontend sends a POST request to /api/rooms/ with room details.
3	The backend processes the request and saves the new room details in the database.
4	The database confirms the successful creation of the room.
5	The backend sends a room creation confirmation response to the frontend.
6	The frontend notifies the admin that the room has been successfully created.

Table 36

Alternative Action Flow

Step	Action
1	If the provided room details are invalid or incomplete, the system returns an error message.
2	The admin is prompted to re-enter the correct details.

Table 37

Admin - Generate Reports

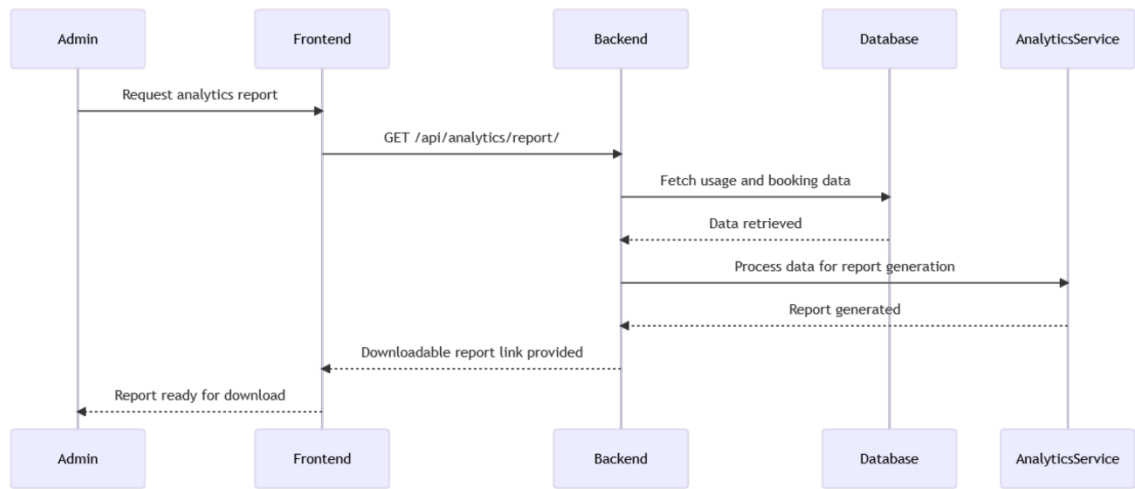


Figure 15

Use Case ID	13
Use Case Name	Generate Reports
Description	Administrators can generate analytics reports on room utilization.
Primary Actor	Admin
Secondary Actor	None
Requirements	Historical data must be available in the system.
Post Action	The report is generated and made available for download.

Table 38

Action Flow

Step	Action
1	The admin selects the reporting module.
2	The admin specifies the report type and filters (e.g., date range, room).
3	The system retrieves the relevant data and processes it.
4	A downloadable report is generated.

Table 39

Alternative Action Flow

Step	Action
1	If no data is available for the specified criteria, the admin is notified.

Table 40

Chatbot Interaction

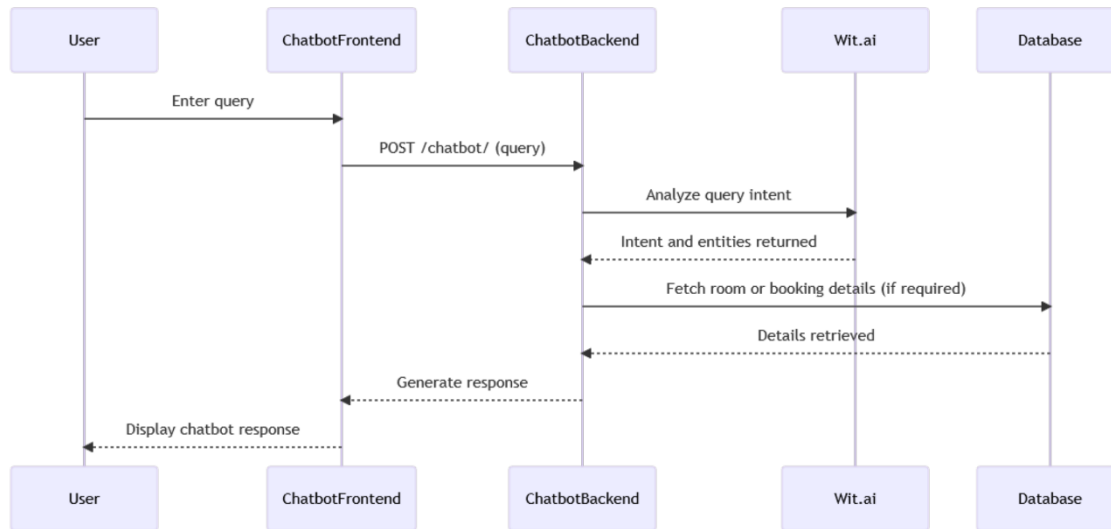


Figure 16

Use Case ID	14
Use Case Name	Chatbot Interaction
Description	Users can interact with a chatbot to inquire about room availability, bookings, or navigation.
Primary Actor	Student, Lecturer
Secondary Actor	None
Requirements	The chatbot must be integrated with Wit.ai for NLP capabilities.
Post Action	The chatbot responds to the user query with relevant information.

Table 41

Action Flow

Step	Action
1	The user enters a query into the chatbot.
2	The chatbot sends the query to Wit.ai for intent recognition.
3	The system processes the intent and retrieves the required data.
4	The chatbot generates a response and displays it to the user.

Table 42

Alternative Action Flow

Step	Action
1	If the query cannot be resolved, the chatbot provides a fallback response.

Table 43

Admin-Deactivate User

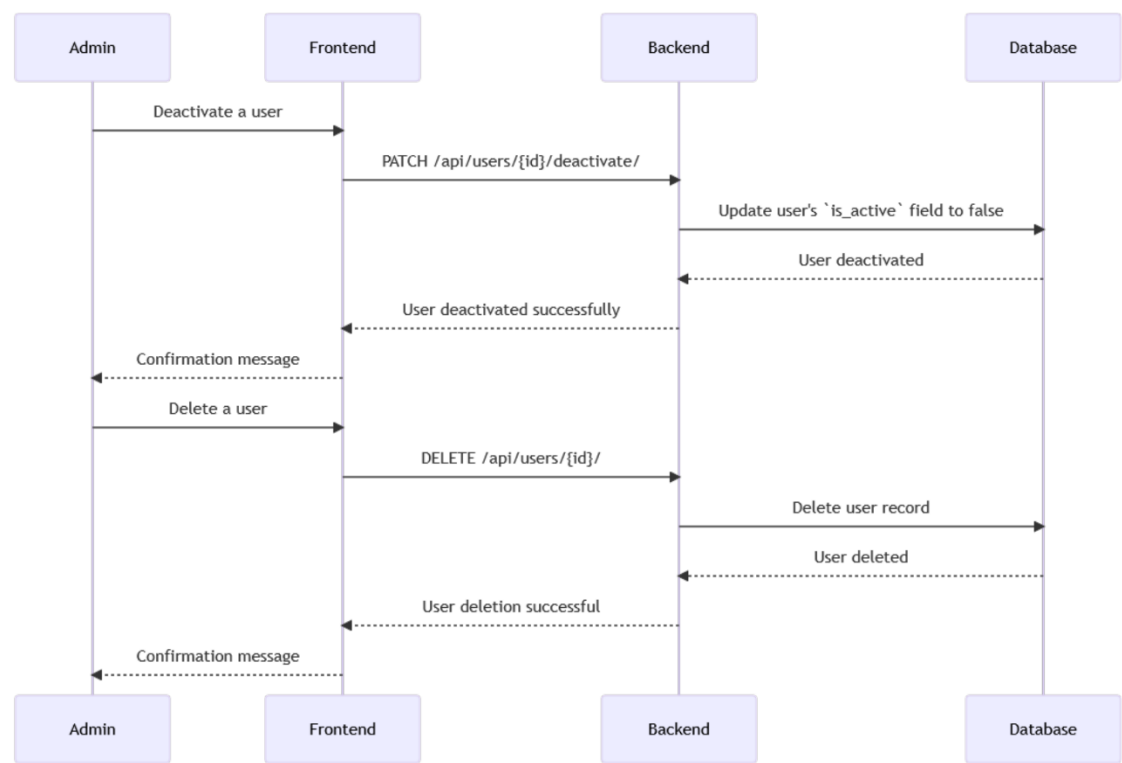


Figure 17

Use Case ID	15
Use Case Name	Admin Can Deactivate or Delete a User
Description	The admin can deactivate or permanently delete a user from the system.
Primary Actor	Admin
Secondary Actor	None
Requirements	The user must exist in the system, and the admin must have the necessary permissions.
Post Action	The user is either deactivated (set to inactive) or permanently deleted from the system.

Table 44

Action Flow

Step	Action
1	The admin selects the option to deactivate or delete a user.
2	If deactivating, the frontend sends a PATCH request to /API/users/{id}/deactivate/.

3	The backend processes the request and updates the is active field to false in the database.
4	The database confirms the user has been deactivated.
5	The backend sends a deactivation success message to the frontend.
6	If deleting, the frontend sends a DELETE request to /API/users/{id}/.
7	The backend processes the request and deletes the user record from the database.
8	The database confirms the user has been deleted.
9	The backend sends a deletion success message to the frontend.
10	The frontend displays a confirmation message to the admin.

Table 45

Alternative Action Flow

Step	Action
1	If the specified user ID does not exist, the system returns an error message.
2	The admin is notified that the user cannot be found or has already been deactivated/deleted.

Table 46

Admin-Create Notification

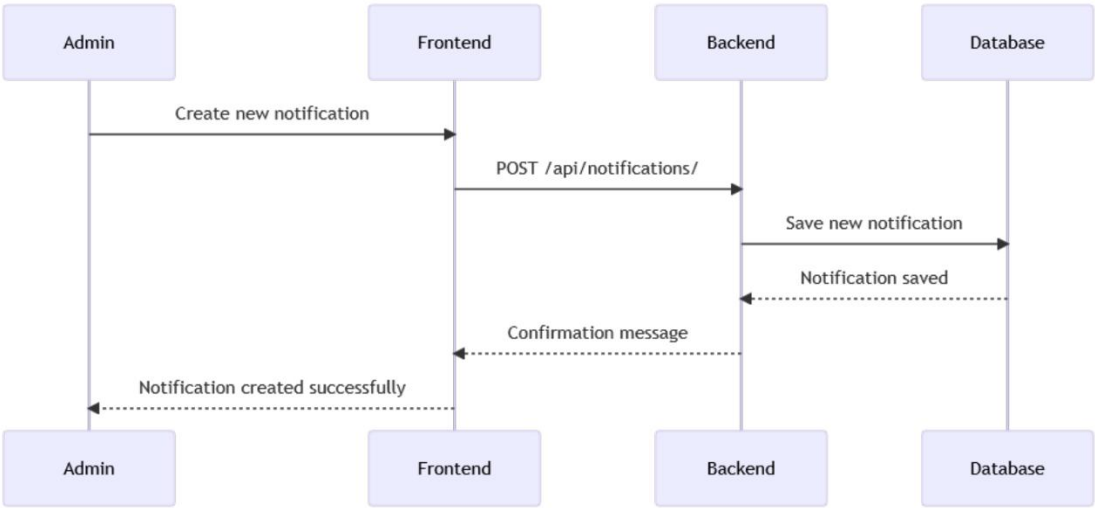


Figure 18

Use Case ID	14 (Example, replace with actual ID)
Use Case Name	Admin Create Notification
Description	The admin can create and send notifications to users through the system.
Primary Actor	Admin
Secondary Actor	None
Requirements	The admin must provide valid notification content (title, message, recipients).
Post Action	The notification is stored in the system and displayed to users.

Table 47

Action Flow

Step	Action
1	The admin creates a new notification using the system interface.
2	The frontend sends a POST request to /api/notifications/ with the notification details.
3	The backend processes the request and saves the notification in the database.
4	The database confirms the notification is saved.
5	The backend sends a confirmation message to the frontend.
6	The frontend notifies the admin that the notification was created successfully.

Table 48

Alternative Action Flow

Step	Action
1	If the notification details are incomplete or invalid, the system returns an error message.
2	The admin is prompted to provide valid notification details.

Table 49

Handling No-show or Late Cancellation Penalty

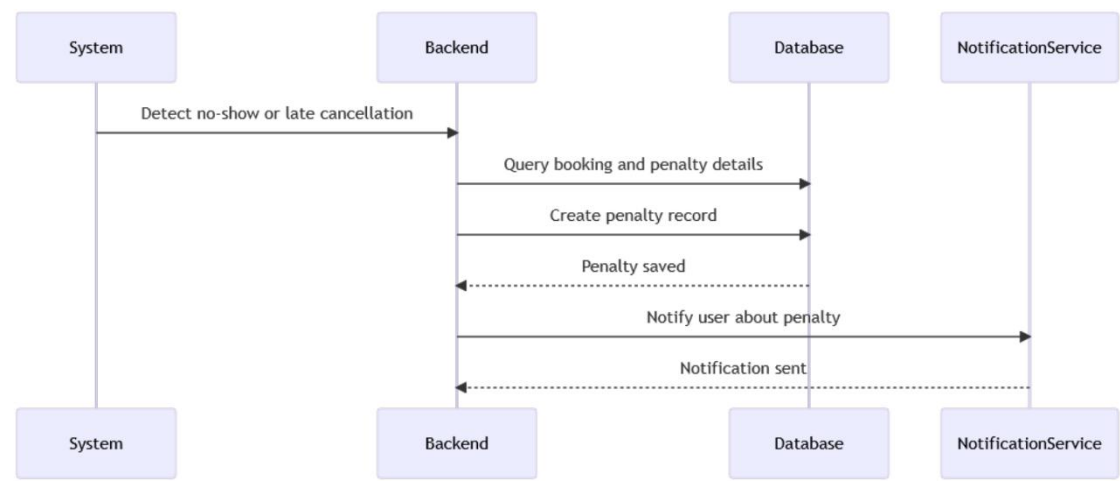


Figure 19

Use Case ID	15 (Example, replace with actual ID)
Use Case Name	Handling No-show or Late Cancellation Penalty
Description	The system detects when a user does not show up for their booking or cancels late and applies a penalty accordingly.
Primary Actor	System (automated process)
Secondary Actor	Notification Service, Admin
Requirements	The booking system must track reservations and detect no-shows or late cancellations based on predefined rules.
Post Action	The penalty is recorded in the system and the user is notified.

Table 50

Action Flow

Step	Action
1	The system detects a no-show or late cancellation.
2	The backend queries the database for booking and penalty details.
3	The system creates a penalty record in the database.
4	The database confirms that the penalty has been saved.

5	The backend notifies the notification service about the penalty.
6	The notification service sends a notification to the user.
7	The system logs the completion of the penalty process.

Table 51

Alternative Action Flow

Step	Action
1	If no booking details are found, an error is logged.
2	If penalty creation fails, the system retries or notifies the admin.
3	If the notification service fails, the system retries sending notifications or logs the failure.

Table 52

Overall - Class Diagram

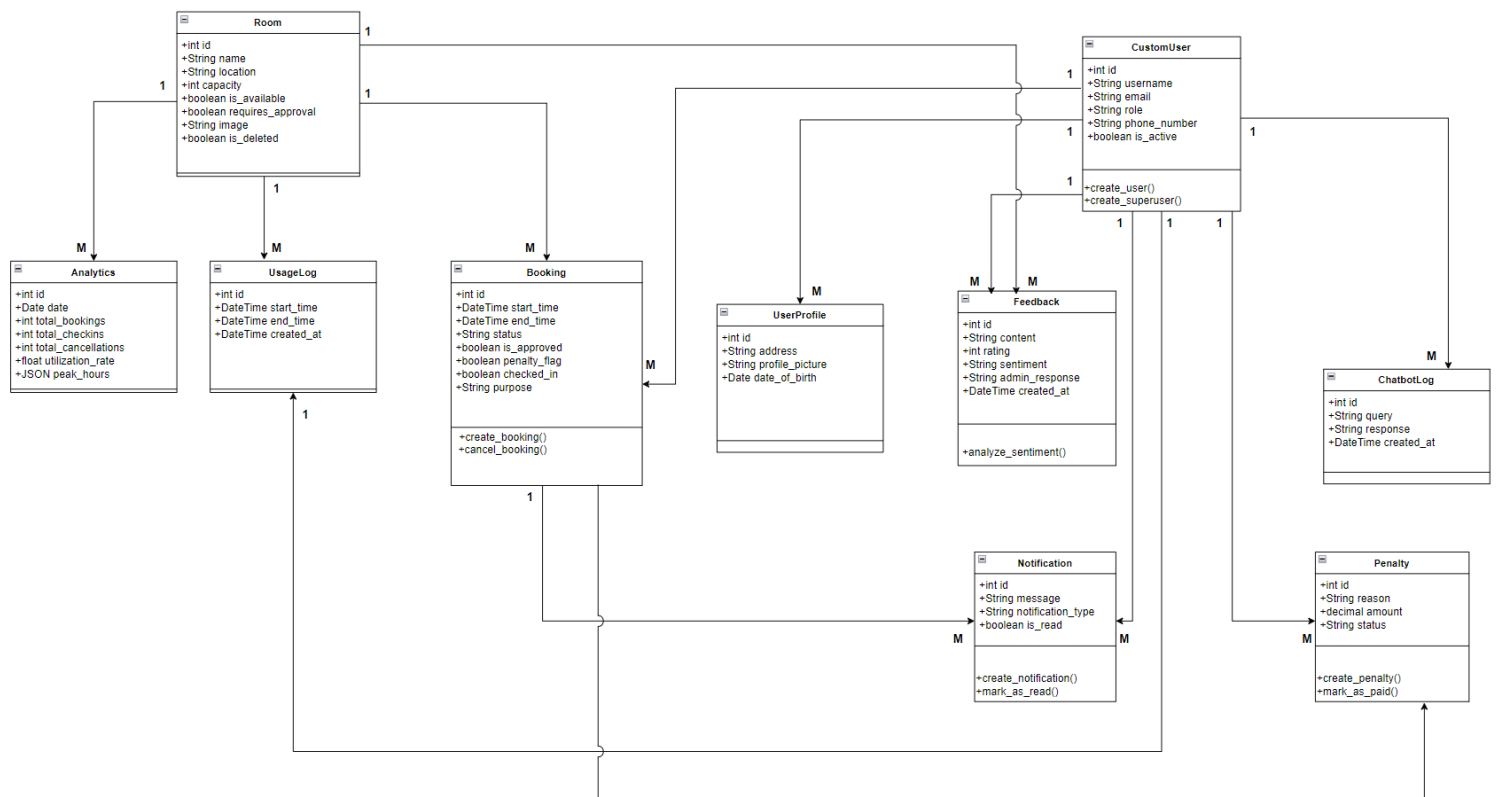


Figure 20

API Specifications

Users App

Endpoint	Request Method	Description
/api/users/	GET, POST	List all users or create a new user.
/api/users/register_user/	POST	Register a new user with specified details.
/api/users/login/	POST	Login user with email and password.
/api/users/profile/	GET, PUT	Get or update the authenticated user's profile.
/api/users/change_password/	POST	Change the authenticated user's password.
/api/users/deactivate_self/	POST	Deactivate the authenticated user's account.
/api/users/deactivate/<id>/	PATCH	Admin action to deactivate a user account.

Rooms App

Endpoint	Request Method	Description
/api/rooms/	GET, POST	List all rooms or create a new room.
/api/rooms/start_usage/<id>/	POST	Start room usage for the authenticated user.
/api/rooms/end_usage/<id>/	POST	End room usage for the authenticated user.

Bookings App

Endpoint	Request Method	Description
/api/bookings/	GET, POST	List all bookings or create a new booking.
/api/bookings/my_bookings/	GET	Get bookings for the currently logged-in user.
/api/bookings/count-pending/	GET	Count the number of pending bookings.
/api/bookings/approve/<id>/	POST	Approve a booking (admin only).
/api/bookings/cancel/<id>/	POST	Cancel a booking (admin only).
/api/bookings/check_in/<id>/	POST	Mark a booking as checked-in.

Chatbot App

Endpoint	Request Method	Description
/chatbot/	POST	Process user input through the chatbot for various intents.

Feedback App

Endpoint	Request Method	Description
/api/feedback/	GET, POST	List all feedback or submit a new feedback.
/api/feedback/stats/	GET	Get feedback statistics for the admin dashboard.
/api/feedback/mark_reviewed/<id>/	POST	Mark specific feedback as reviewed.
/api/feedback/report/	GET	Generate a downloadable feedback report in CSV format.

Notifications App

Endpoint	Request Method	Description
/api/notifications/	GET, POST	List all notifications for the user or create a new one.

/api/notifications/my_notifications/	GET	Retrieve notifications for the logged-in user.
/api/notifications/mark_as_read/<id>/	POST	Mark a specific notification as read.
/api/notifications/mark_all_as_read/	POST	Mark all notifications for the user as read.

Penalties App

Endpoint	Request Method	Description
/api/penalties/	GET, POST	List all penalties or create a new penalty.
/api/penalties/user_penalties/	GET	Get penalties for the currently logged-in user.
/api/penalties/request_review/<id>/	POST	Submit a review request for a penalty.
/api/penalties/resolve/<id>/	POST	Resolve a penalty (admin only).
/api/penalties/download_report/	GET	Generate a penalties report in CSV format.

Analytics App

Endpoint	Request Method	Description
/api/analytics/	GET, POST	List analytics data or create new analytics records.
/api/analytics/chart_data/	GET	Retrieve analytics data for frontend charts.
/api/analytics/export_csv/	GET	Export analytics data in CSV format.

Conclusion

Achieving success at ECU Sri Lanka (ECUSL) The Room Booking System (RBS) has effectively improved both process efficiency and optimization for lecture hall and study room bookings. The automation of manual booking procedures creates a central system that shows real-time availability while enforcing access rules and penalties against violators. Through the development and implementation of RBS, significant operational efficiencies have been realized, including:

- The new system forecasts a 60% drop in administration work requirements so employees can shift their attention from manual bookings toward essential responsibilities.
- Real-time availability tracking combined with automatic notifications creates 50% improved room utilization because they eliminate unused bookings.
- Smoother operations with enhanced user satisfaction emerge from the 30% reduction in booking conflicts that RBS provides with the system.
- The implementation would produce expected savings of 40% that stemmed from reducing paperwork expenses combined with phone-based inquiries and approval procedures run by personnel.
- The system enables nearly full accuracy levels in penalty applications which increases both booking policy fairness and decreases challenges between users and managers.

The system supports scalable operations through modern web technologies which combine industry's best practices to deliver secure user-friendly interfaces that create a foundation for continued growth and development.

Technical Achievements

The project successfully delivered the following technical outcomes:

1. **Automated Booking Workflow:**
 - End-to-end room booking processes, including real-time availability checks, approval workflows, and penalty enforcement.
 - Role-based access ensures that students, staff, and administrators have tailored access to features.
2. **Data Analytics and Reporting:**
 - Advanced analytics modules provide detailed insights into room usage trends, peak utilization, and penalty statistics.
 - Exportable reports in CSV format allow for administrative reviews and further data processing.
3. **Scalable Architecture:**
 - A modular backend powered by Django ensures maintainability and scalability for future enhancements.
 - Integration with external tools like Wit.ai for chatbot functionality showcases flexibility in adapting new technologies.
4. **User-Centric Design:**
 - A user-friendly interface simplifies complex workflows, reducing learning curves for all roles.
 - Feedback and sentiment analysis provide actionable insights to improve user experience.

Challenges Addressed

- **Real-time Synchronization:** Managing room statuses dynamically was critical to avoid double bookings.
 - *Solution:* Optimized database queries and implemented validation checks at the API level.
- **Penalty Enforcement Logic:** Automating penalties for no-shows and late cancellations required precise time validations.
 - *Solution:* Developed workflows to calculate and impose penalties based on booking and usage logs.
- **Scalability Concerns:** Ensuring the system could handle multiple concurrent users across roles without degradation in performance.
 - *Solution:* Introduced caching mechanisms and asynchronous task handling where appropriate.

Key Findings

- **Operational Efficiency:** Automation of room booking and management significantly reduces the administrative burden.
- **Improved Accountability:** Penalty systems and user notifications promote responsible resource usage.
- **Enhanced Accessibility:** The chatbot provides seamless interaction, ensuring inclusivity and 24/7 assistance.

Future Technical Enhancements

To maximize the system's potential, the following improvements are recommended:

1. **Enhanced Predictive Analytics:**
 - Utilize machine learning models for forecasting room demand based on historical usage patterns.
2. **Mobile Integration:**
 - Develop native mobile applications to enhance accessibility and user engagement.
3. **Dynamic Dashboards:**
 - Introduce real-time visual analytics for administrators to monitor room usage and user activity.
4. **API Extensions:**
 - Expand APIs to support third-party integrations, enabling interoperability with other university systems.

Final Statement

The Room Booking System serves as a critical step toward digitizing campus operations. By addressing real-world challenges with a highly modular, scalable, and secure architecture, the system provides a foundation for future technological advancements. This project not only fulfills the defined requirements but also lays the groundwork for continuous innovation, ensuring long-term sustainability and alignment with institutional goals.

References

- Administrative Financial Planning and Analysis (AFPA) department of the University of Cambridge. (2013). Review of Room Booking Systems. In *University of Cambridge*.
<https://www.afpa.admin.cam.ac.uk/>
- Leary, C. (2023, February 2). Facility space management Strategies in higher education. *BP Logix*.
<https://www.bplogix.com/blog/facility-space-management-strategies-in-higher-education>
- *Design and development of an integrated room reservation system for higher education institutions*. (2021, April 23). IEEE Conference Publication | IEEE Xplore.
<https://ieeexplore.ieee.org/document/9436766>
- IEOM Society. (2020). EReserba Cardinal: an integrated room reservation system for higher education institutions. *Proceedings of the International Conference on Industrial Engineering and Operations Management (IEOM)*.
<https://ieomsociety.org/ieom2020/papers/362.pdf>

- -THE END - -